REVISION CHEAT SHEET: ASP.NET CORE / STARTUP

ASP.NET Core Fundamentals and Application Startup

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. What Is ASP.NET Core?
2. ASP.NET Core Is Part of .NET
3. What Happens When an ASP.NET Core App Starts?
Diagram 1 — ASP.NET Core Startup
4. Program.cs Is the Composition Root
5. Top-Level Statements
6. Traditional Main Equivalent
7. WebApplication.CreateBuilder(args)
8. WebApplicationBuilder Mental Model
9. Important Builder Properties
builder.Services
builder.Configuration
builder.Environment
builder.Logging
builder.Host
builder.WebHost
10. What Is the Host?
11. WebApplication vs Generic Host
12. WebApplication Is the Modern Web Host Model
13. What About Startup.cs?
14. Understanding the Two Startup Phases
Diagram 2 — Builder Phase vs App Phase
15. builder.Build()
16. WebApplication
17. app.Run()
18. What Happens During Run?
19. What Is Kestrel?
20. Kestrel Flow
21. Kestrel and Reverse Proxies
22. Configuring Server URLs
23. launchSettings.json
24. launchSettings.json vs appsettings.json
launchSettings.json
appsettings.json
25. ASP.NET Core Configuration
26. IConfiguration
27. appsettings.json
28. Environment-Specific Configuration
29. Default Configuration Priority
Diagram 3 — Configuration Override Flow
30. Why Environment Variables Are Important
31. Double Underscore in Environment Variables
32. Connection Strings
33. Do Not Store Production Secrets in appsettings.json
34. User Secrets
35. Strongly Typed Configuration
36. Better Options Registration
37. Why Fail Fast?
38. IOptions<T>
39. Runtime Environments
40. Why Environments Exist
41. Checking Environment
42. Environment Determines Configuration File
43. Environment Variables for Environment Name
44. Development Exception Information
45. Dependency Injection During Startup
46. Startup Extensions Keep Program.cs Manageable
47. Infrastructure Registration Extension
48. Registering Controllers
49. Minimal APIs
50. Controllers vs Minimal APIs

51. Service Registration vs Middleware Registration
52. Some Middleware Is Added Automatically
53. Logging Fundamentals
54. Structured Logging
55. Log Levels
56. Logging Categories
57. Built-In Logging Providers
58. Do Not Log Sensitive Information
59. Logging Is Synchronous at the API Surface
60. Application Lifetime
61. ApplicationStarted
62. ApplicationStopping
63. ApplicationStopped
Diagram 4 — Host Lifetime
64. Graceful Shutdown
65. Environment.Exit vs Graceful Shutdown
66. Hosted Services
67. BackgroundService
68. Startup Work Must Be Controlled
69. Database Migration at Startup
70. Startup Seeding
71. Startup Failure Is Often Correct
72. But Do Not Require Every External Service to Be Online at Startup
73. Health Checks vs Startup
74. Content Root
75. Web Root
76. Dependency Graph at Startup
77. A Clean Fundraising Program.cs
78. Program.cs Should Tell the Architecture Story
79. Startup Extension Naming
80. Configuration Should Not Become Service Locator
81. Environment Checks Should Not Spread Everywhere
82. Example Environment-Specific Service Registration
83. Startup Logging
84. Command-Line Configuration
85. Configuration Should Describe Environment, Not Business State
86. Startup and Performance
87. DI Resolution Performance
88. Configuration Reload
89. Startup Security Checklist
90. Never Trust Devel... Production
91. Common Mistake — Registering Services After Build
92. Common Mistake — Business Logic in Program.cs
93. Common Mistake — IConfiguration Everywhere
94. Common Mistake — Production Secrets in JSON
95. Common Mistake — launchSettings.json as Production Configuration
96. Common Mistake — Environment-Specific Business Logic Everywhere
97. Common Mistake — Blocking Async Startup
98. Common Mistake — Starting Background Threads Manually
99. Common Mistake — Giant Program.cs
100. Common Mistake — Catching Startup Exceptions and Pretending Everything Is Fine
101. Best Practices
PART 2 — COMPLETE FUNDRAISING STARTUP EXAMPLE
102. appsettings.json
103. ReceiptOptions
104. Application Registration
105. Infrastructure Registration
106. Complete Program.cs
107. Understanding This Program Line by Line
Diagram 5 — Complete Fundraising Startup Flow
PART 3 — PRACTICAL EXERCISES
Exercise 1 — Minimal Application
Exercise 2 — Configuration Priority
Exercise 3 — Strongly Typed Options
Exercise 4 — Fail-Fast Configuration
Exercise 5 — Environment
Exercise 6 — DI Registration
Exercise 7 — Startup Extensions
Exercise 8 — Kestrel URL
Exercise 9 — Logging
Exercise 10 — User Secrets
Exercise 11 — Hosted Service
Exercise 12 — Graceful Shutdown
PART 4 — EXACTLY 25 INTERVIEW QUESTIONS
1. What is ASP.NET Core?
2. What is Program.cs in an ASP.NET Core application?
3. What does WebApplication.CreateBuilder(args) do?
4. What is WebApplicationBuilder?

5. What does builder.Build() do?
6. What does app.Run() do?
7. What is the Generic Host?
8. What is the difference between WebApplication and the older WebHost?
9. What is Kestrel?
10. What is the difference between AddControllers and MapControllers?
11. What is IConfiguration?
12. What is the default configuration precedence?
13. What is appsettings.json?
14. What are appsettings.Development.json and appsettings.Production.json?
15. What are ASP.NET Core runtime environments?
16. What is launchSettings.json?
17. What is the Options
ASP.NET CORE / STARTUP
18. What does ValidateOnStart do?
19. Where should application secrets be stored?
20. What is ILogger<T> ?
21. What is structured logging?
22. What is IHostApplicationLifetime?
23. What is IHostedService?
24. Why should Program.cs not contain business logic?
25. How would startup work in the Fundraising Platform?
QUICK REVISION
FINAL END-TO-END CONCEPT FLOW

1. What Is ASP.NET Core?

ASP.NET Core is Microsoft's modern, cross-platform framework for building web applications and HTTP services on .NET.

Common workloads include:

- REST APIs
- Minimal APIs
- MVC applications
- Razor Pages
- SignalR applications
- gRPC services
- Background-hosted web workloads

For your Fundraising Platform, the primary workload is:

EXAMPLE / FLOW

```
Angular / Client
      ↓
HTTP / HTTPS
      ↓
ASP.NET Core Web API
      ↓
Application Layer
      ↓
Domain
      ↓
EF Core
      ↓
SQL Server
```

ASP.NET Core provides much more than HTTP routing. The application host also coordinates configuration, dependency injection, logging, application lifetime, hosted services, and the HTTP server. Microsoft describes the host as the object that encapsulates these application resources and manages startup and graceful shutdown.

2. ASP.NET Core Is Part of .NET

Keep these concepts separate:

EXAMPLE / FLOW

```
.NET
    ↓
Runtime + base libraries + SDK + tools
```

EXAMPLE / FLOW

```
ASP.NET Core
    ↓
Web framework built on .NET
```

EXAMPLE / FLOW

```
EF Core
    ↓
Object-relational mapper
```

EXAMPLE / FLOW

```
C#
    ↓
Programming language
```

A typical Fundraising API therefore uses:

EXAMPLE / FLOW

```
C# 14
   ↓
.NET 10
   ↓
ASP.NET Core 10
   ↓
EF Core
   ↓
SQL Server
```

3. What Happens When an ASP.NET Core App Starts?

A modern ASP.NET Core application usually begins with:

C#

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet(
    "/",
    () => "Hello World!");

app.Run();
```

This tiny program hides a substantial amount of framework setup.

Conceptually:

EXAMPLE / FLOW

```
Process starts
    ↓
Create WebApplicationBuilder
    ↓
Load host configuration
    ↓
```

```
Load app configuration
    ↓
Set environment
    ↓
Configure logging
    ↓
Create service collection
    ↓
Register application services
    ↓
Build application
    ↓
Create DI service provider
    ↓
Configure middleware/endpoints
    ↓
Run host
    ↓
Start hosted services
    ↓
Start Kestrel
    ↓
Listen for HTTP requests
```

`WebApplicationBuilder` and `WebApplication` are the recommended hosting types used by current ASP.NET Core project templates.

Diagram 1 — ASP.NET Core Startup

EXAMPLE / FLOW

```
dotnet Fundraising.Api.dll
          ↓
Program.cs
          ↓
WebApplication.CreateBuilder(args)
          ↓
┌──────────────────────────────────┐
│ Configuration                    │
│ Logging                          │
│ Environment                      │
│ Dependency Injection             │
│ Web server configuration         │
└──────────────────────────────────┘
          ↓
Register Services
          ↓
builder.Build()
          ↓
Configure Request Pipeline
          ↓
Map Endpoints
          ↓
app.Run()
          ↓
Hosted services start
          ↓
Kestrel starts listening
          ↓
Application ready
```

Microsoft's current startup documentation describes this sequence as hosted-service startup, middleware pipeline finalization, web-server startup, application-started lifetime notificat… host waiting until shutdown.

4. Program.cs Is the Composition Root

In modern ASP.NET Core, most application startup configuration begins in:

EXAMPLE / FLOW

```
Program.cs
```

`Program.cs` is usually your application's **composition root**.

That means it is where you wire together:

EXAMPLE / FLOW

```
Configuration
Dependency Injection
Database
Authentication
Authorization
Logging
HTTP clients
Application services
Infrastructure services
Middleware
Endpoints
Hosted services
```

It should configure the system.

It should not contain business logic.

Bad:

C#

```
if (gift.TotalAmount > 10_000m)
{
    gift.MarkAsMajor();
}
```

inside `Program.cs`.

Good:

C#

```
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

5. Top-Level Statements

Modern C# allows the template to use:

C#

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();
```

```
app.Run();
```


without explicitly writing:

```
C#
public static class Program
{
    public static void Main(
        string[] args)
    {
    }
}
```

The compiler generates the entry-point structure.

Therefore:

EXAMPLE / FLOW

```
Program.cs
```

still represents the program entry point even when you cannot see an explicit `Main` method.

6. Traditional Main Equivalent

Conceptually:

```
C#
public static class Program
{
    public static void Main(
        string[] args)
    {
        WebApplicationBuilder builder =
            WebApplication.CreateBuilder(
                args);

        WebApplication app = builder.Build();

        app.Run();
    }
}
```

Top-level statements merely make this more concise.

7. WebApplication.CreateBuilder(args)

This is one of the most important calls:

```
C#
var builder = WebApplication.CreateBuilder(args);
```

It creates a:

```
C#
WebApplicationBuilder
```

with framework defaults already configured.

Microsoft documents builder properties including configuration, environment, hosting, logging, metrics, service registration, and web-host/server configuration.

Important properties include:

```
C#
builder.Configuration

builder.Environment

builder.Services

builder.Logging

builder.Host

builder.WebHost
```

8. WebApplicationBuilder Mental Model

Think of the builder as the:

> **Pre-startup configuration object.**

Before the application is built:

```
EXAMPLE / FLOW
WebApplicationBuilder
        ↓
Configure everything needed to build host
```

Examples:

```
C#
builder.Services.AddControllers();

builder.Services.AddDbContext<
    FundraisingDbContext>();

builder.Services.AddScoped<
    IGiftRepository,
    GiftRepository>();

builder.Services.AddAuthentication();

builder.Services.AddAuthorization();
```

You are describing:

```
EXAMPLE / FLOW
"These services and behaviors should exist
when the application starts."
```

9. Important Builder Properties

builder.Services

Type:

C#
```
IServiceCollection
```

Purpose:

EXAMPLE / FLOW
```
Register dependencies
```

Example:

C#
```
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

builder.Configuration

Provides application configuration.

Example:

C#
```
string? connectionString =
    builder.Configuration
        .GetConnectionString(
            "FundraisingDb");
```

builder.Environment

Provides runtime-environment information.

Example:

C#
```
if (
    builder.Environment
        .IsDevelopment())
{
    // Development-only service setup.
}
```

builder.Logging

Configures logging providers and behavior.

Example:

C#
```
builder.Logging
    .AddConsole();
```

builder.Host

Provides lower-level Generic Host configuration.

builder.WebHost

Provides web-host/server-specific configuration.

For normal applications, most configuration should use the simpler direct builder properties unless a lower-level hosting feature is genuinely required.

10. What Is the Host?

The **host** is the infrastructure responsible for running the application.

The Generic Host manages resources such as:

EXAMPLE / FLOW

```
Dependency Injection
Configuration
Logging
Hosted services
Application lifetime
Graceful shutdown
```

In a web application it also participates in hosting the HTTP server.

Microsoft describes lifetime management as a primary reason these resources are grouped together under the host.

11. WebApplication vs Generic Host

Modern web application:

C#

```
WebApplication.CreateBuilder(args)
```

Non-web/worker-style Generic Host:

C#

```
Host.CreateApplicationBuilder(args)
```

Older hosting style:

C#

```
Host.CreateDefaultBuilder(args)
```

Current .NET guidance recommends the property-based modern host-builder style for new applications, while the older `IHostBuilder` callback model remains supported mainly for compatibility and existing applications.

12. WebApplication Is the Modern Web Host Model

Microsoft's current ASP.NET Core templates use:

EXAMPLE / FLOW

```
WebApplicationBuilder
+
WebApplication
```

because this hosting model requires less startup ceremony than older ASP.NET Core hosting approaches.

For a new .NET 10 API, think:

EXAMPLE / FLOW

```
WebApplication.CreateBuilder
        ↓
WebApplicationBuilder
        ↓
Build
```

```
        ↓
WebApplication
```

13. What About Startup.cs?

Older ASP.NET Core applications commonly used:

EXAMPLE / FLOW

```
Program.cs
Startup.cs
```

with methods such as:

C#

```
ConfigureServices(...)
```

and:

C#

```
Configure(...)
```

Current templates do not require a `Startup` class.

Modern startup is typically:

C#

```
var builder = WebApplication.CreateBuilder(args);

// Configure services.

var app = builder.Build();

// Configure middleware/endpoints.

app.Run();
```

Microsoft describes `WebApplicationBuilder` and `WebApplication` as the streamlined modern approach that configures web applications without requiring a `Startup` class.

14. Understanding the Two Startup Phases

A very important mental model is:

EXAMPLE / FLOW

```
BEFORE Build()
    ↓
Configure services / host
```

and:

EXAMPLE / FLOW

```
AFTER Build()
    ↓
Configure HTTP application pipeline
```

Example:

C#

```
var builder = WebApplication.CreateBuilder(args);

// PHASE 1
builder.Services.AddControllers();

builder.Services
    .AddScoped<
        IGiftService,
        GiftService>();

var app = builder.Build();

// PHASE 2
app.UseHttpsRedirection();

app.MapControllers();

app.Run();
```

Diagram 2 — Builder Phase vs App Phase

EXAMPLE / FLOW

```
PHASE 1 — BUILDING
──────────────────────────────────

WebApplicationBuilder

builder.Configuration
builder.Services
builder.Logging
builder.Environment
builder.WebHost

        ↓

builder.Build()


PHASE 2 — RUNNING APP CONFIGURATION
──────────────────────────────────────

WebApplication

app.Use...
app.Map...
app.Run()
```

A useful simplified rule:

> **Register services before `Build`; configure the HTTP pipeline and endpoints after `Build`.**

15. builder.Build()

Calling:

C#

```
var app = builder.Build();
```

creates the configured:

```C#
WebApplication
```

and establishes the application service provider/object graph.

After this point, startup code transitions from:

```EXAMPLE / FLOW
service registration
```

to:

```EXAMPLE / FLOW
HTTP application configuration
```

Microsoft's startup documentation states that `WebApplicationBuilder.Build()` returns the built `WebApplication`, after which the request-handling pipeline is established.

16. WebApplication

`WebApplication` represents the built web application.

You use it for things such as:

```C#
app.UseHttpsRedirection();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

app.MapGet(...);

app.Run();
```

It combines several responsibilities convenient for web startup, including endpoint mapping and host execution.

17. app.Run()

At the end of startup:

```C#
app.Run();
```

starts the host and keeps the process running until shutdown.

Microsoft's startup documentation states that `Run` starts the application and blocks the calling thread while the host remains active.

You can also use asynchronous host APIs where appropriate:

```C#
await app.RunAsync();
```

18. What Happens During Run?

Current ASP.NET Core startup behavior can be understood as:

EXAMPLE / FLOW

```
app.Run()
    ↓
Start hosted services
    ↓
Finalize middleware pipeline
    ↓
Start configured HTTP server
    ↓
Kestrel binds sockets/ports
    ↓
ApplicationStarted fired
    ↓
Host remains running
```

Microsoft documents hosted-service startup followed by pipeline finalization and server startup as part of the startup lifecycle.

19. What Is Kestrel?

Kestrel is ASP.NET Core's cross-platform web server.

It:

EXAMPLE / FLOW

```
Listens on network ports

Accepts HTTP connections

Parses HTTP requests

Passes requests into ASP.NET Core

Writes HTTP responses
```

Kestrel is the recommended server and is configured by default in ASP.NET Core project templates.

20. Kestrel Flow

EXAMPLE / FLOW

```
Browser / Angular
       ↓
TCP connection
       ↓
Kestrel
       ↓
ASP.NET Core Request Pipeline
       ↓
Endpoint
       ↓
Response
       ↓
Kestrel
       ↓
Client
```

Kestrel is not:

EXAMPLE / FLOW

```
your controller
your middleware
your application service
```

It is the HTTP server hosting the application.

21. Kestrel and Reverse Proxies

Production deployment may look like:

EXAMPLE / FLOW

```
Internet
    ↓
Load Balancer / Reverse Proxy
    ↓
Kestrel
    ↓
ASP.NET Core
```

Examples of front-end infrastructure can include:

EXAMPLE / FLOW

```
IIS
Nginx
Apache
Cloud load balancer
Ingress controller
```

Kestrel can also be internet-facing depending on deployment architecture.

22. Configuring Server URLs

Kestrel endpoints can be configured through several mechanisms, including configuration, environment variables, command-line arguments, and code. Microsoft documents mechanisms such as `ASPNETCORE_URLS`, `--urls`, configuration keys, and explicit Kestrel endpoint configuration.

Example command:

C#

```
dotnet run --urls=http://localhost:8080
```

Environment variable concept:

EXAMPLE / FLOW

```
ASPNETCORE_URLS=http://localhost:8080
```

23. launchSettings.json

A development project typically contains:

EXAMPLE / FLOW

```
Properties/
    launchSettings.json
```

Example:

C#

```
{
  "profiles": {
    "https": {
      "commandName": "Project",
      "launchBrowser": true,
      "applicationUrl":
        "https://localhost:7045;http://localhost:5045",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT":
          "Development"
      }
    }
  }
}
```

Important:

> `launchSettings.json` is local development tooling configuration.

It is not your production application configuration and is not deployed with the published application.

24. launchSettings.json vs appsettings.json

Do not confuse them.

launchSettings.json

Used mainly by development tooling for:

EXAMPLE / FLOW

```
Launch profiles
Local URLs
Environment variables
Browser launch
```

appsettings.json

Used by the application configuration system for:

EXAMPLE / FLOW

```
Logging
Connection-string references
Feature settings
Application options
Integration URLs
```

25. ASP.NET Core Configuration

ASP.NET Core uses a configuration system based on:

EXAMPLE / FLOW

```
Key
    →
Value
```

Configuration may come from:

- JSON files

- Environment variables
- Command-line arguments
- User Secrets
- Azure Key Vault
- Azure App Configuration
- In-memory providers
- Custom providers

Microsoft's configuration framework combines these providers into one configuration model.

26. IConfiguration

Configuration can be accessed through:

```C#
IConfiguration
```

In `Program.cs`:

```C#
IConfiguration configuration = builder.Configuration;
```

Read:

```C#
string? value =
    configuration[
        "Receipts:Sender"];
```

Hierarchical key:

```EXAMPLE / FLOW
Receipts:Sender
```

represents:

```C#
{
  "Receipts": {
    "Sender": "Email"
  }
}
```

27. appsettings.json

Example:

```C#
{
  "ConnectionStrings": {
    "FundraisingDb":
      "Server=.;Database=FundraisingDb;Integrated Security=true;"
  },

  "Receipt": {
    "FromName":
      "Fundraising Platform",
```

```
    "TimeoutSeconds":
      30
  },

  "Logging": {
    "LogLevel": {
      "Default":
        "Information",

      "Microsoft.AspNetCore":
        "Warning"
    }
  }
}
```

28. Environment-Specific Configuration

Typical files:

EXAMPLE / FLOW

```
appsettings.json

appsettings.Development.json

appsettings.Staging.json

appsettings.Production.json
```

Suppose:

C#

```
// appsettings.json
{
  "Receipt": {
    "TimeoutSeconds": 30
  }
}
```

and:

C#

```
// appsettings.Development.json
{
  "Receipt": {
    "TimeoutSeconds": 5
  }
}
```

Development resolves:

EXAMPLE / FLOW

```
Receipt:TimeoutSeconds
    =
5
```

because the environment-specific provider is loaded after the base JSON provider and therefore overrides matching keys.

29. Default Configuration Priority

A highly important concept is:

> **When multiple providers contain the same key, the higher-priority/later effective provider wins.**

For the default ASP.NET Core application configuration, the practical precedence from highest to lower priority is:

EXAMPLE / FLOW

```
Command-line arguments
        ↓
Environment variables
        ↓
User Secrets in Development
        ↓
appsettings.{Environment}.json
        ↓
appsettings.json
        ↓
fallback host configuration
```

Microsoft documents this default ordering for `WebApplication.CreateBuilder`.

Diagram 3 — Configuration Override Flow

EXAMPLE / FLOW

```
appsettings.json
       ↓ overridden by
appsettings.Development.json
       ↓ overridden by
User Secrets
       ↓ overridden by
Environment Variables
       ↓ overridden by
Command-Line Arguments
       ↓
Final IConfiguration value
```

This enables one application binary to run in many environments without recompilation.

30. Why Environment Variables Are Important

Suppose:

EXAMPLE / FLOW

```
Development
    →
local SQL Server
```

and:

EXAMPLE / FLOW

```
Production
    →
production SQL Server
```

Do not compile different source code.

Instead configuration changes:

EXAMPLE / FLOW
```
ConnectionStrings__FundraisingDb
```

Environment variables can override configuration loaded from JSON. Microsoft recommends using environment variables or secure external configuration rather than putting production secrets directly into source-controlled settings files.

31. Double Underscore in Environment Variables

JSON hierarchy uses:

EXAMPLE / FLOW
```
:
```

For example:

EXAMPLE / FLOW
```
Receipt:TimeoutSeconds
```

Environment variables commonly use:

EXAMPLE / FLOW
```
__
```

instead:

EXAMPLE / FLOW
```
Receipt__TimeoutSeconds
```

because `:` isn't portable across all shells/platforms.

So:

EXAMPLE / FLOW
```
ConnectionStrings__FundraisingDb
```

maps to:

EXAMPLE / FLOW
```
ConnectionStrings:FundraisingDb
```

32. Connection Strings

Typical configuration:

C#
```
{
  "ConnectionStrings": {
    "FundraisingDb":
      "..."
  }
}
```

Access:

```C#
string connectionString =
    builder.Configuration
        .GetConnectionString(
            "FundraisingDb")
    ??
    throw new
        InvalidOperationException(
            "FundraisingDb connection string is missing.");
```

Registration:

```C#
builder.Services
    .AddDbContext<
        FundraisingDbContext>(
            options =>
                options.UseSqlServer(
                    connectionString));
```

33. Do Not Store Production Secrets in appsettings.json

Bad:

```C#
{
  "Payment": {
    "ApiKey":
      "real-production-secret"
  }
}
```

especially if the file is committed.

Microsoft recommends never storing passwords or other sensitive information in plain-text configuration files or provider code and recommends keeping production secrets outside the project.

34. User Secrets

During local development, .NET provides the **Secret Manager** mechanism.

Conceptually:

```EXAMPLE / FLOW
Source-controlled appsettings.json
       ↓
contains normal settings

Developer Secret Store
       ↓
contains local secrets
```

For example:

```EXAMPLE / FLOW
ConnectionStrings:FundraisingDb

Payment:ApiKey
```

User Secrets are a **development convenience**, not a production secret-management architecture.

For production, use secure external secret infrastructure appropriate to the hosting platform. Microsoft specifically identifies Secret Manager for Development and recommends secure stores such as Key Vault for production scenarios.

35. Strongly Typed Configuration

Instead of repeatedly writing:

```
C#
configuration[
    "Receipt:TimeoutSeconds"]
```

create:

```
C#
public sealed class ReceiptOptions
{
    public const string SectionName = "Receipt";

    public required string FromName
    {
        get;
        init;
    }

    public int TimeoutSeconds
    {
        get;
        init;
    }
}
```

Register:

```
C#
builder.Services
    .Configure<
        ReceiptOptions>(
            builder.Configuration
                .GetSection(
                    ReceiptOptions.SectionName));
```

The Options pattern provides strongly typed access to related settings and improves encapsulation and separation of concerns.

36. Better Options Registration

A useful production-oriented pattern is:

```
C#
builder.Services
    .AddOptions<
        ReceiptOptions>()
    .Bind(
        builder.Configuration
            .GetSection(
                ReceiptOptions.SectionName))
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

`ValidateOnStart()` causes invalid options to fail during application startup instead of waiting until the setting is first used.

This is often preferable for required configuration.

37. Why Fail Fast?

Suppose:

EXAMPLE / FLOW

```
Payment:BaseUrl
```

is required.

Without startup validation:

EXAMPLE / FLOW

```
App starts successfully
        ↓
receives production request
        ↓
payment client needs BaseUrl
        ↓
fails
```

With startup validation:

EXAMPLE / FLOW

```
App starts
    ↓
configuration validated
    ↓
missing setting detected
    ↓
startup fails immediately
```

This is called:

EXAMPLE / FLOW

```
fail fast
```

and is usually better for mandatory infrastructure configuration.

38. IOptions<T>

Example consumer:

C#

```
public sealed class ReceiptSender
{
    private readonly
        ReceiptOptions
        _options;

    public ReceiptSender(
        IOptions<
            ReceiptOptions>
            options)
    {
        _options = options.Value;
```

```
    }
}
```


Other Options abstractions include:

EXAMPLE / FLOW

```
IOptions<T>

IOptionsSnapshot<T>

IOptionsMonitor<T>
```

You already encountered their DI-lifetime implications in Topic 32.

At startup-design level, remember:

EXAMPLE / FLOW

```
Options
    =
strongly typed configuration boundary
```

39. Runtime Environments

ASP.NET Core supports runtime environment names.

Framework-provided conventional names are:

EXAMPLE / FLOW

```
Development

Staging

Production
```

but custom environment names are possible.

If the environment isn't configured, ASP.NET Core defaults to:

EXAMPLE / FLOW

```
Production
```

The selected environment remains fixed for the application's lifetime.

40. Why Environments Exist

Different environments require different behavior.

Development might use:

EXAMPLE / FLOW

```
Detailed exception information

Local database

Verbose logging

Development integrations
```

Production might use:

EXAMPLE / FLOW

```
Production monitoring

Production database

Restricted error responses

Secure external secrets

Performance optimizations
```

Do not write:

C#

```
#if DEBUG
```

for ordinary runtime environment behavior when the runtime environment system is the appropriate tool.

41. Checking Environment

Before Build:

C#

```
if (
    builder.Environment
        .IsDevelopment())
{
}
```

After Build:

C#

```
if (
    app.Environment
        .IsDevelopment())
{
}
```

Other helpers include:

C#

```
IsProduction()

IsStaging()

IsEnvironment("QA")
```

Microsoft recommends using `builder.Environment` while services are being configured and `app.Environment` while configuring the built application.

42. Environment Determines Configuration File

Suppose:

EXAMPLE / FLOW

```
Environment = Production
```

then configuration includes:

EXAMPLE / FLOW

```
appsettings.json
+
appsettings.Production.json
```

Suppose:

EXAMPLE / FLOW

```
Environment = Development
```

then:

EXAMPLE / FLOW

```
appsettings.json
+
appsettings.Development.json
```

Environment-specific values override matching base settings.

43. Environment Variables for Environment Name

ASP.NET Core hosting recognizes:

EXAMPLE / FLOW

```
DOTNET_ENVIRONMENT

ASPNETCORE_ENVIRONMENT
```

for environment selection.

The exact precedence depends on the hosting API being used, so modern applications should avoid setting contradictory values and should inspect:

C#

```
builder.Environment.EnvironmentName
```

as the authoritative runtime result.

If neither environment variable is set, the default environment is Production.

44. Development Exception Information

Development often exposes detailed diagnostics useful to developers.

Production should not expose internal stack traces or exception details to anonymous clients.

Why?

Stack traces can reveal:

EXAMPLE / FLOW

```
Source structure

Class names

SQL details

File paths

Infrastructure information
```

```
Internal implementation
```


Detailed developer diagnostics belong in trusted development scenarios.

Topic 38 will cover exception-handling middleware in detail.

45. Dependency Injection During Startup

ASP.NET Core includes a built-in dependency injection container.

Register services:

```C#
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        CreateGiftService>();

builder.Services
    .AddSingleton<
        IClock,
        SystemClock>();
```

Then consumers declare their dependencies:

```C#
public sealed class CreateGiftService
{
    private readonly
        IGiftRepository
        _repository;

    public CreateGiftService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

ASP.NET Core resolves the object graph automatically.

46. Startup Extensions Keep Program.cs Manageable

Large applications should avoid a 1,500-line `Program.cs`.

Instead:

```C#
builder.Services
    .AddFundraisingApplication();

builder.Services
    .AddFundraisingInfrastructure(
        builder.Configuration);

builder.Services
    .AddFundraisingApi();
```

Then extension methods organize registration.

Example:

```C#
public static class
    ApplicationServiceCollectionExtensions
{
    public static IServiceCollection
        AddFundraisingApplication(
            this IServiceCollection services)
    {
        services.AddScoped<
            CreateGiftService>();

        services.AddScoped<
            CompleteGiftService>();

        return services;
    }
}
```

47. Infrastructure Registration Extension

```C#
public static class
    InfrastructureServiceCollectionExtensions
{
    public static IServiceCollection
        AddFundraisingInfrastructure(
            this IServiceCollection services,
            IConfiguration configuration)
    {
        string connectionString =
            configuration
                .GetConnectionString(
                    "FundraisingDb")
            ??
            throw new
                InvalidOperationException(
                    "Missing FundraisingDb connection string.");

        services.AddDbContext<
            FundraisingDbContext>(
                options =>
                    options.UseSqlServer(
                        connectionString));

        services.AddScoped<
            IGiftRepository,
            GiftRepository>();

        return services;
    }
}
```

This keeps the composition root readable without hiding the architectural registration boundaries.

48. Registering Controllers

For controller-based Web APIs:

```
C#
builder.Services
    .AddControllers();
```

registers services required by MVC/controller infrastructure.

Later:

```
C#
app.MapControllers();
```

maps attribute-routed controller endpoints.

These are two different phases:

```
EXAMPLE / FLOW
AddControllers()
    →
service registration
```

```
EXAMPLE / FLOW
MapControllers()
    →
endpoint registration
```

49. Minimal APIs

An ASP.NET Core application can also define endpoints directly:

```
C#
app.MapGet(
    "/api/gifts/{giftId:long}",
    async (
        long giftId,
        GiftQueries queries,
        CancellationToken cancellationToken) =>
    {
        GiftResponse? gift =
            await queries.GetAsync(
                giftId,
                cancellationToken);

        return gift is null
            ? Results.NotFound()
            : Results.Ok(
                gift);
    });
```

Minimal APIs use:

```
EXAMPLE / FLOW
WebApplication
+
route handlers
```

and are first-class ASP.NET Core APIs.

50. Controllers vs Minimal APIs

Controllers provide features such as:

```
EXAMPLE / FLOW
Controller classes

Attributes

Action methods

Filters

Conventions
```

Minimal APIs favor:

```
EXAMPLE / FLOW
Lower ceremony

Direct route handlers

Compact endpoint definitions
```

Neither is automatically better.

For a larger business API such as the Fundraising Platform, controllers or feature-organized Minimal APIs can both work.

Architecture matters more than syntax.

51. Service Registration vs Middleware Registration

Do not confuse:

```
C#
builder.Services.AddAuthentication();
```

with:

```
C#
app.UseAuthentication();
```

The first:

```
EXAMPLE / FLOW
registers services
```

The second:

```
EXAMPLE / FLOW
participates in HTTP processing
```

Similarly:

```
C#
builder.Services.AddAuthorization();
```

configures authorization services.

```C#
app.UseAuthorization();
```

puts authorization behavior into the relevant request pipeline.

Middleware ordering becomes the focus of Topic 38.

52. Some Middleware Is Added Automatically

The modern `WebApplication` hosting model can automatically add certain framework middleware depending on what is configured—for example routing, authentication, authorization, and the developer exception page in applicable cases.

However:

> ⓘ Do not assume middleware order never matters.

Applications often explicitly call middleware methods when ordering relative to other middleware matters.

For example:

```C#
app.UseCors();

app.UseAuthentication();

app.UseAuthorization();
```

Microsoft specifically documents cases where explicit ordering is required.

Topic 38 will cover the pipeline thoroughly.

53. Logging Fundamentals

ASP.NET Core has built-in logging abstractions:

```C#
ILogger

ILogger<T>

ILoggerFactory
```

Example:

```C#
public sealed class GiftService
{
    private readonly
        ILogger<GiftService>
        _logger;

    public GiftService(
        ILogger<GiftService> logger)
    {
        _logger = logger;
    }
}
```

Usage:

```
C#
_logger.LogInformation(
    "Processing Gift {GiftId}",
    giftId);
```

54. Structured Logging

Prefer:

```
C#
_logger.LogInformation(
    "Gift {GiftId} completed for Tenant {TenantId}",
    giftId,
    tenantId);
```

rather than:

```
C#
_logger.LogInformation(
    $"Gift {giftId} completed for Tenant {tenantId}");
```

Structured placeholders preserve named values for logging systems.

Conceptually:

```
EXAMPLE / FLOW
Message template:
Gift {GiftId} completed for Tenant {TenantId}

Properties:
GiftId = 123
TenantId = 10
```

This helps querying and observability.

55. Log Levels

Built-in levels include:

```
EXAMPLE / FLOW
Trace
Debug
Information
Warning
Error
Critical
None
```

A configured minimum level determines which messages are processed.

Microsoft documents this ordering and notes that messages below the configured minimum aren't passed to providers.

56. Logging Categories

`ILogger<T>` uses the type as a category.

Example:

C#
```
ILogger<GiftService>
```

category roughly corresponds to:

EXAMPLE / FLOW
```
YourNamespace.GiftService
```

Configuration can set different levels for categories:

C#
```
{
  "Logging": {
    "LogLevel": {
      "Default":
        "Information",

      "Microsoft.AspNetCore":
        "Warning",

      "Fundraising":
        "Debug"
    }
  }
}
```

ASP.NET Core logging configuration supports category-based filtering.

57. Built-In Logging Providers

ASP.NET Core includes providers such as:

EXAMPLE / FLOW
```
Console

Debug

EventSource

Windows EventLog
```

Additional providers can be installed for other destinations.

The abstraction:

EXAMPLE / FLOW
```
ILogger
```

allows application code to remain independent of the final log destination.

58. Do Not Log Sensitive Information

Avoid logging:

EXAMPLE / FLOW
```
Passwords

API secrets

Access tokens
```

```
Full payment-card data

Sensitive personal data

Raw authorization headers
```

Logging is an external data boundary.

A log line may be copied into:

EXAMPLE / FLOW

```
Central log stores

Monitoring systems

Support tools

Long-retention archives
```

Treat it accordingly.

59. Logging Is Synchronous at the API Surface

There is no normal:

C#

```
LogInformationAsync(...)
```

API.

Microsoft's logging guidance notes that logging methods are designed to remain fast and that slow log destinations shouldn't be synchronously written directly from application logging calls.

Do not implement:

EXAMPLE / FLOW

```
every log
    ↓
synchronous SQL INSERT
```

on the request thread.

60. Application Lifetime

The host manages application startup and shutdown.

Important abstraction:

C#

```
IHostApplicationLifetime
```

It exposes cancellation tokens for:

EXAMPLE / FLOW

```
ApplicationStarted

ApplicationStopping

ApplicationStopped
```

and provides:

```csharp
StopApplication()
```

for requesting graceful termination.

61. ApplicationStarted

`ApplicationStarted` fires when the host has fully started.

Conceptually:

```
Services ready
    ↓
Server started
    ↓
ApplicationStarted
```

Useful for rare infrastructure scenarios requiring notification that startup is complete.

Don't use it as a general replacement for ordinary DI initialization.

62. ApplicationStopping

Triggered when graceful shutdown begins.

At this point:

```
Shutdown has started

Requests may still be in flight

Hosted services are being stopped
```

Use it for bounded cleanup notification where appropriate.

63. ApplicationStopped

Triggered after graceful shutdown has completed.

Microsoft documents the host lifecycle and the ordering of hosted-service/lifetime notifications around startup and shutdown.

Diagram 4 — Host Lifetime

```
Process starts
     ↓
Build host
     ↓
Hosted services StartAsync
     ↓
Server starts
     ↓
ApplicationStarted
     ↓
_____________________
APPLICATION RUNNING
```

```
SIGTERM / Ctrl+C / StopApplication
     ↓
ApplicationStopping
     ↓
Server stops accepting new work
     ↓
Hosted services StopAsync
     ↓
ApplicationStopped
     ↓
Process exits
```

64. Graceful Shutdown

In production, processes may stop because of:

EXAMPLE / FLOW

```
Deployment

Container replacement

Server restart

Ctrl+C

SIGTERM

Host shutdown
```

A graceful shutdown gives the application an opportunity to:

EXAMPLE / FLOW

```
stop accepting new requests

finish in-flight work when possible

stop hosted services

release resources
```

The Generic Host coordinates this lifecycle.

65. Environment.Exit vs Graceful Shutdown

Avoid:

C#

```
Environment.Exit(1);
```

as the normal host shutdown mechanism.

When application code intentionally wants the host to stop, the host lifetime abstraction provides:

C#

```
IHostApplicationLifetime
    .StopApplication();
```

which requests graceful shutdown. Microsoft explicitly recommends graceful host shutdown rather than directly terminating the process when possible.

66. Hosted Services

A service implementing:

```
C#
IHostedService
```

participates in host startup/shutdown.

Registration:

```
C#
builder.Services
    .AddHostedService<
        OutboxBackgroundService>();
```

The host invokes hosted-service startup when the application starts.

This is useful for:

```
EXAMPLE / FLOW
Outbox publishing

Background queue processing

Scheduled maintenance

Long-running workers
```

67. BackgroundService

A convenient base class is:

```
C#
BackgroundService
```

Example:

```
C#
public sealed class OutboxBackgroundService :
    BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (
            !stoppingToken
                .IsCancellationRequested)
        {
            await ProcessAsync(
                stoppingToken);

            await Task.Delay(
                TimeSpan.FromSeconds(5),
                stoppingToken);
        }
```

```
    }
}
```

ASP.NET CORE / STARTUP

The cancellation token signals application shutdown.

Remember from Topic 32:

```
EXAMPLE / FLOW
Hosted service
    =
singleton
```

so scoped dependencies such as `DbContext` should be resolved through an explicit scope.

68. Startup Work Must Be Controlled

Bad:

```
C#
var builder = WebApplication.CreateBuilder(args);

var client = new HttpClient();

var data =
    client
        .GetStringAsync(
            "https://remote-service")
        .Result;
```

Problems:

```
EXAMPLE / FLOW
Sync-over-async

Remote dependency during composition

Uncontrolled startup delay

Hard testing

Unclear failure behavior
```

Do not fill `Program.cs` with arbitrary business/integration work.

Use deliberate startup initialization or hosted services when necessary.

69. Database Migration at Startup

Some projects do:

```
C#
using IServiceScope scope =
    app.Services
        .CreateScope();

FundraisingDbContext dbContext =
    scope.ServiceProvider
        .GetRequiredService<
            FundraisingDbContext>();
```

```
await dbContext.Database
    .MigrateAsync();
```

ASP.NET CORE / STARTUP

This can be convenient in development or controlled deployments.

But production architecture must consider:

EXAMPLE / FLOW

```
Multiple app instances starting together

Migration permissions

Long-running migrations

Deployment rollback

Schema compatibility

Downtime
```

Automatically applying production migrations from every app instance is not universally appropriate.

70. Startup Seeding

Similarly:

EXAMPLE / FLOW

```
Database seeding
```

may be useful for:

EXAMPLE / FLOW

```
Development reference data

Test fixtures

Required fixed system data
```

but shouldn't become uncontrolled production business-data creation.

Keep migrations, reference-data initialization, and application business operations conceptually separate.

71. Startup Failure Is Often Correct

Suppose the application requires:

EXAMPLE / FLOW

```
Database connection string

Payment provider base URL

Signing certificate
```

and one is missing.

You usually want:

EXAMPLE / FLOW

```
Application fails startup
```

instead of:

EXAMPLE / FLOW

```
Application reports "healthy"
    ↓
first real request fails
```

Startup options validation is one mechanism for enforcing this fail-fast approach.

72. But Do Not Require Every External Service to Be Online at Startup

There is an important distinction.

Bad startup dependency:

EXAMPLE / FLOW

```
App won't start unless
email provider currently responds.
```

Transient third-party outages may then prevent deployment.

Separate:

EXAMPLE / FLOW

```
Configuration validity
```

from:

EXAMPLE / FLOW

```
Current dependency availability
```

For example:

EXAMPLE / FLOW

```
Payment BaseUrl must exist
```

can be startup validation.

But:

EXAMPLE / FLOW

```
Payment server must answer every deployment
```

may be better represented through health checks/resilience rather than hard startup dependency.

73. Health Checks vs Startup

Health checks answer questions such as:

EXAMPLE / FLOW

```
Is application alive?

Is application ready?

Can dependency X be reached?
```

They are operational runtime mechanisms.

Startup validation asks:

EXAMPLE / FLOW

```
Can this application be correctly constructed?
```

These are related but not identical.

74. Content Root

The host has a:

EXAMPLE / FLOW

```
ContentRootPath
```

which identifies the base path used for application content/configuration.

`IHostEnvironment` provides information such as:

EXAMPLE / FLOW

```
ApplicationName

EnvironmentName

ContentRootPath
```

and `IWebHostEnvironment` additionally provides web-specific information such as the web root.

75. Web Root

The web root is typically associated with:

EXAMPLE / FLOW

```
wwwroot
```

for publicly served static web assets.

For a pure API, you may barely use `wwwroot`.

For server-rendered/static-content applications, it becomes more relevant.

76. Dependency Graph at Startup

Suppose:

EXAMPLE / FLOW

```
GiftController
    ↓
CreateGiftService
    ↓
IGiftRepository
    ↓
GiftRepository
    ↓
FundraisingDbContext
```

Registration:

C#

```
builder.Services.AddControllers();

builder.Services
    .AddScoped<
```

```
        CreateGiftService>();

builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddDbContext<
        FundraisingDbContext>(
            options =>
                options.UseSqlServer(
                    connectionString));
```

At runtime the DI container can construct:

EXAMPLE / FLOW

```
GiftController
      ↓
CreateGiftService
      ↓
GiftRepository
      ↓
FundraisingDbContext
```

This is why startup configuration is tightly connected to application architecture.

77. A Clean Fundraising Program.cs

A practical structure could be:

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddControllers();

builder.Services
    .AddOpenApi();

builder.Services
    .AddFundraisingApplication();

builder.Services
    .AddFundraisingInfrastructure(
        builder.Configuration);

builder.Services
    .AddFundraisingAuthentication(
        builder.Configuration);

builder.Services
    .AddOptions<
        ReceiptOptions>()
    .Bind(
        builder.Configuration
            .GetSection(
                ReceiptOptions.SectionName))
    .ValidateDataAnnotations()
    .ValidateOnStart();

var app = builder.Build();
```

```
if (
    app.Environment
        .IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

The exact registrations depend on your application, but the structure is important.

78. Program.cs Should Tell the Architecture Story

A developer should be able to open `Program.cs` and roughly see:

EXAMPLE / FLOW

```
What framework features exist

What application modules exist

What infrastructure is registered

What middleware runs

What endpoints are exposed
```

Avoid:

EXAMPLE / FLOW

```
thousands of low-level registrations
```

but also avoid:

EXAMPLE / FLOW

```
one mysterious AddEverything()
```

that hides the entire application.

Find a meaningful abstraction level.

79. Startup Extension Naming

Good:

C#

```
AddFundraisingApplication()

AddFundraisingInfrastructure()

AddFundraisingAuthentication()

AddPaymentIntegration()
```

These describe capabilities.

Less useful:

```
C#
AddStuff()

AddHelpers()

ConfigureEverything()
```

Startup organization should communicate architectural boundaries.

80. Configuration Should Not Become Service Locator

Bad:

```
C#
public sealed class GiftService
{
    private readonly
        IConfiguration
        _configuration;

    public GiftService(
        IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public void Process()
    {
        string? timeout =
            _configuration[
                "Payment:Timeout"];
    }
}
```

Now the service can read every setting in the application.

Prefer:

```
C#
public GiftService(
    IOptions<
        PaymentOptions>
        options)
```

when the class needs a cohesive configuration group.

This makes dependencies explicit.

81. Environment Checks Should Not Spread Everywhere

Bad:

```
C#
if (
    environment.IsProduction())
{
```

```
    // in 80 business classes...
}
```

Business behavior should normally be controlled through configuration/DI abstractions.

For example:

EXAMPLE / FLOW

```
Development
    →
FakePaymentGateway

Production
    →
RealPaymentGateway
```

can be expressed through DI registration rather than environment `if` statements inside the domain.

82. Example Environment-Specific Service Registration

C#

```
if (
    builder.Environment
        .IsDevelopment())
{
    builder.Services
        .AddScoped<
            IPaymentGateway,
            DevelopmentPaymentGateway>();
}
else
{
    builder.Services
        .AddScoped<
            IPaymentGateway,
            PaymentGateway>();
}
```

Now consuming code only knows:

C#

```
IPaymentGateway
```

This preserves separation of concerns.

83. Startup Logging

The host provides startup logging automatically, such as:

EXAMPLE / FLOW

```
Hosting environment

Content root

Listening URLs
```

Application-specific startup logs can also be useful.

Do not log:

EXAMPLE / FLOW

```
connection-string passwords
API keys
secret configuration
```

while debugging startup problems.

84. Command-Line Configuration

Example:

C#

```
dotnet run -- \
  Receipt:TimeoutSeconds=60
```

Command-line configuration has high priority and normally overrides values from JSON, user secrets, and environment variables.

This can be useful in:

EXAMPLE / FLOW

```
Containers

Automation

Temporary testing

Deployment scripts
```

but excessive command-line configuration can become difficult to manage.

85. Configuration Should Describe Environment, Not Business State

Good configuration:

EXAMPLE / FLOW

```
Payment API URL

Timeout

Retry count

Feature toggle

Connection string
```

Poor configuration:

EXAMPLE / FLOW

```
Current Gift

Current user

Tenant's current donation amount
```

Configuration is not a substitute for application/database state.

86. Startup and Performance

Startup performance matters in:

EXAMPLE / FLOW

```
Containers

Autoscaling

Serverless-like environments

Rapid deployment

Health-based orchestration
```

Avoid expensive synchronous startup work such as:

EXAMPLE / FLOW

```
Loading millions of rows

Scanning entire file systems

Calling ten external APIs

Building enormous caches unnecessarily
```

Some initialization may need to be lazy or handled as background work.

87. DI Resolution Performance

Normal DI resolution is typically not your Web API's major bottleneck.

Much larger costs usually come from:

EXAMPLE / FLOW

```
Database queries

Network calls

Serialization

Large allocations

Excessive logging

Poor caching strategy
```

Do not convert everything to singleton merely to avoid object construction.

Correct lifetime semantics matter more.

88. Configuration Reload

Some configuration providers support reload behavior.

However, not every consumer sees updates the same way.

Remember:

EXAMPLE / FLOW

```
IOptions<T>
    →
```

```
stable cached option instance

IOptionsSnapshot<T>
    →
scoped/request snapshot

IOptionsMonitor<T>
    →
can observe/currently retrieve updates
```

Dynamic configuration should be intentional.

Do not assume changing:

EXAMPLE / FLOW

```
appsettings.json
```

means every production deployment will safely update business behavior instantly.

89. Startup Security Checklist

At startup, verify architectural decisions around:

EXAMPLE / FLOW

```
HTTPS

Authentication

Authorization

Secret storage

Allowed hosts/proxy setup

Configuration validation

Logging redaction

Production error handling

Database privileges

External-service credentials
```

The goal is:

EXAMPLE / FLOW

```
secure defaults
```

rather than remembering security individually in every controller.

90. Never Trust Development Configuration in Production

Development may contain:

EXAMPLE / FLOW

```
Verbose diagnostics

Local credentials

Mock services
```

```
Development CORS settings

Local endpoints

Relaxed certificates
```

Production should not accidentally inherit unsafe development behavior.

Use:

```
EXAMPLE / FLOW
environment-specific configuration
+
startup validation
+
deployment configuration
```

to keep the boundary explicit.

91. Common Mistake — Registering Services After Build

Bad mental model:

```
EXAMPLE / FLOW
Build application
    ↓
then decide what services exist
```

Registration belongs in the builder phase.

Prefer:

```
C#
builder.Services
    .AddScoped<
        GiftService>();

var app = builder.Build();
```

not attempts to modify the core service collection after it has already been used to build the application provider.

92. Common Mistake — Business Logic in Program.cs

Bad:

```
C#
Gift gift =
    await dbContext.Gifts
        .SingleAsync(...);

gift.Complete();

await dbContext
    .SaveChangesAsync();
```

inside startup.

`Program.cs` should compose the application, not execute normal business use cases.

93. Common Mistake — IConfiguration Everywhere

Bad:

```
EXAMPLE / FLOW
Every service
    ↓
IConfiguration
```

Now every class can read every setting.

Prefer:

```
EXAMPLE / FLOW
narrow strongly typed options
```

for cohesive settings.

94. Common Mistake — Production Secrets in JSON

Bad:

```
EXAMPLE / FLOW
appsettings.Production.json
    ↓
committed real database password
```

Environment-specific JSON is still usually source-controlled application configuration.

It is not automatically a secret store.

95. Common Mistake — launchSettings.json as Production Configuration

`launchSettings.json` is local development tooling configuration and isn't included in published deployment output.

Do not expect:

```
EXAMPLE / FLOW
production server
```

to use your Visual Studio launch profile.

96. Common Mistake — Environment-Specific Business Logic Everywhere

Bad:

```
C#
if (
    env.IsProduction())
{
    CalculateFeeA();
}
else
{
```

```
    CalculateFeeB();
}
```


Business policy should normally be represented as:

```
EXAMPLE / FLOW
configuration
strategy
service implementation
domain policy
```

rather than being tied directly to deployment environment names.

97. Common Mistake — Blocking Async Startup

Bad:

```
C#
var data =
    service.LoadAsync()
        .Result;
```

Use deliberate asynchronous lifecycle mechanisms where initialization genuinely requires asynchronous work.

Never introduce sync-over-async merely because startup code feels “special.”

98. Common Mistake — Starting Background Threads Manually

Bad:

```
C#
new Thread(
    () =>
        ProcessOutbox())
    .Start();
```

ASP.NET Core already has host-managed background-service abstractions.

Use:

```
EXAMPLE / FLOW
IHostedService

BackgroundService
```

so work participates in application startup and graceful shutdown.

99. Common Mistake — Giant Program.cs

Bad:

```
EXAMPLE / FLOW
Program.cs
    =
1,800 lines
```

with every registration expanded manually.

Use cohesive registration extensions.

But avoid hiding important architecture behind an unreadable single call.

100. Common Mistake — Catching Startup Exceptions and Pretending Everything Is Fine

Bad:

C#

```
try
{
    ConfigureCriticalDatabase();
}
catch
{
    // Ignore.
}
```

Then:

EXAMPLE / FLOW

```
Application appears healthy
but cannot function.
```

If a dependency/configuration is required for correctness, fail startup clearly.

101. Best Practices

Use the modern `WebApplicationBuilder` / `WebApplication` hosting model for new ASP.NET Core applications.

Keep `Program.cs` as the composition root.

Register services before `builder.Build()`.

Configure the HTTP pipeline and endpoints after `Build()`.

Use DI rather than manually constructing service graphs.

Use strongly typed Options for cohesive configuration.

Validate required configuration at startup.

Keep secrets outside source-controlled configuration files.

Use environment-specific settings only for genuine environmental differences.

Keep business rules out of startup code.

Use structured logging.

Do not log secrets.

Use Kestrel configuration appropriate to deployment.

Use host-managed `IHostedService` / `BackgroundService` for long-running background work.

Design graceful shutdown behavior.

Avoid long synchronous work during startup.

Fail fast on invalid mandatory configuration.

Keep startup organization aligned with your architectural modules.

PART 2 — COMPLETE FUNDRAISING STARTUP EXAMPLE

102. appsettings.json

C#

```
{
  "ConnectionStrings": {
    "FundraisingDb":
      ""
  },

  "Receipt": {
    "FromName":
      "Fundraising Platform",

    "TimeoutSeconds":
      30
  },

  "Payment": {
    "BaseUrl":
      "https://payments.example.com"
  },

  "Logging": {
    "LogLevel": {
      "Default":
        "Information",

      "Microsoft.AspNetCore":
        "Warning"
    }
  }
}
```

Production secret values should come from a secure external configuration source rather than being hard-coded here.

103. ReceiptOptions

C#

```
using System.ComponentModel.DataAnnotations;

public sealed class ReceiptOptions
{
    public const string SectionName = "Receipt";

    [Required]
    public string FromName
    {
        get;
        init;
    } = string.Empty;

    [Range(
        1,
        300)]
    public int TimeoutSeconds
    {
        get;
        init;
```

```
    }
}
```


104. Application Registration

C#

```
public static class
    ApplicationRegistration
{
    public static IServiceCollection
        AddFundraisingApplication(
            this IServiceCollection services)
    {
        services.AddScoped<
            CreateGiftService>();

        services.AddScoped<
            CompleteGiftService>();

        services.AddScoped<
            GiftQueries>();

        return services;
    }
}
```

105. Infrastructure Registration

C#

```
public static class
    InfrastructureRegistration
{
    public static IServiceCollection
        AddFundraisingInfrastructure(
            this IServiceCollection services,
            IConfiguration configuration)
    {
        string connectionString =
            configuration
                .GetConnectionString(
                    "FundraisingDb")
            ??
            throw new
                InvalidOperationException(
                    "ConnectionStrings:FundraisingDb is required.");

        services.AddDbContext<
            FundraisingDbContext>(
                options =>
                    options.UseSqlServer(
                        connectionString));

        services.AddScoped<
            IGiftRepository,
            GiftRepository>();

        services.AddScoped<
            IOutboxWriter,
            OutboxWriter>();

        services.AddSingleton<
```

```
            IClock,
            SystemClock>();

        services.AddHostedService<
            OutboxBackgroundService>();

        return services;
    }
}
```

106. Complete Program.cs

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddControllers();

builder.Services
    .AddOpenApi();

builder.Services
    .AddFundraisingApplication();

builder.Services
    .AddFundraisingInfrastructure(
        builder.Configuration);

builder.Services
    .AddOptions<
        ReceiptOptions>()
    .Bind(
        builder.Configuration
            .GetSection(
                ReceiptOptions.SectionName))
    .ValidateDataAnnotations()
    .ValidateOnStart();

var app = builder.Build();

if (
    app.Environment
        .IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

107. Understanding This Program Line by Line

C#
```
var builder = WebApplication.CreateBuilder(args);
```

creates the host/application builder and loads default hosting infrastructure.

C#
```
builder.Services
    .AddControllers();
```

adds controller/MVC services.

C#
```
builder.Services
    .AddFundraisingApplication();
```

registers use-case/application services.

C#
```
builder.Services
    .AddFundraisingInfrastructure(
        builder.Configuration);
```

registers:

EXAMPLE / FLOW
```
DbContext
Repositories
Clock
Outbox worker
```

C#
```
builder.Services
    .AddOptions<ReceiptOptions>()
```

creates strongly typed configuration.

C#
```
.ValidateOnStart();
```

causes invalid mandatory Receipt configuration to fail startup.

C#
```
var app = builder.Build();
```

builds the application.

C#
```
if (
    app.Environment
        .IsDevelopment())
```

applies Development-specific startup behavior.

C#
```
app.MapControllers();
```

maps controller endpoints.

C#
```
app.Run();
```

starts the host/server and keeps it running.

Diagram 5 — Complete Fundraising Startup Flow

EXAMPLE / FLOW

```
PROCESS START
    ↓
Program.cs
    ↓
WebApplication.CreateBuilder(args)
    ↓
Load Configuration
    │
    ├─ appsettings.json
    ├─ environment JSON
    ├─ User Secrets
    ├─ Environment Variables
    └─ Command Line
    ↓
Determine Environment
    ↓
Configure Logging
    ↓
Register Services
    │
    ├─ Controllers
    ├─ Application Services
    ├─ DbContext
    ├─ Repositories
    ├─ Options
    └─ Background Services
    ↓
Validate Options
    ↓
builder.Build()
    ↓
Configure HTTP Pipeline
    ↓
Map Controllers
    ↓
app.Run()
    ↓
Hosted Services Start
    ↓
Kestrel Starts
    ↓
ApplicationStarted
    ↓
READY FOR REQUESTS
```

PART 3 — PRACTICAL EXERCISES

Exercise 1 — Minimal Application

Create:

C#
```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet(
    "/",
    () =>
        "Fundraising API");

app.Run();
```

Identify:

EXAMPLE / FLOW
```
Builder phase

Build phase

Endpoint phase

Run phase
```

Exercise 2 — Configuration Priority

Set:

EXAMPLE / FLOW
```
Receipt:TimeoutSeconds = 30
```

in `appsettings.json`.

Set:

EXAMPLE / FLOW
```
Receipt:TimeoutSeconds = 5
```

in `appsettings.Development.json`.

Then override with an environment variable.

Observe which value is finally resolved.

Exercise 3 — Strongly Typed Options

Create:

C#
```
PaymentOptions
```

containing:

EXAMPLE / FLOW
```
BaseUrl

TimeoutSeconds
```

Bind:

EXAMPLE / FLOW

```
Payment
```

configuration and validate it at startup.

Exercise 4 — Fail-Fast Configuration

Remove:

EXAMPLE / FLOW

```
ConnectionStrings:FundraisingDb
```

and ensure startup fails with a useful error rather than waiting for the first API request.

Exercise 5 — Environment

Run the application in:

EXAMPLE / FLOW

```
Development

Staging

Production
```

and log:

C#

```
builder.Environment
    .EnvironmentName;
```

Observe which environment-specific JSON file overrides configuration.

Exercise 6 — DI Registration

Register:

EXAMPLE / FLOW

```
IGiftRepository
    →
GiftRepository
```

as scoped.

Inject it into:

C#

```
CreateGiftService
```

and verify the service can be resolved.

Exercise 7 — Startup Extensions

Move Application registrations into:

C#

```
AddFundraisingApplication()
```

and Infrastructure registrations into:

```
C#
AddFundraisingInfrastructure()
```

Keep `Program.cs` readable.

Exercise 8 — Kestrel URL

Run the API on:

```
EXAMPLE / FLOW
http://localhost:8080
```

using command-line URL configuration.

Observe Kestrel's startup log.

Exercise 9 — Logging

Inject:

```
C#
ILogger<CreateGiftService>
```

and log:

```
EXAMPLE / FLOW
GiftId
TenantId
```

using structured placeholders.

Do not use string interpolation.

Exercise 10 — User Secrets

Move a local development API key out of:

```
EXAMPLE / FLOW
appsettings.json
```

into the development secret store.

Verify `IConfiguration` still resolves the same key.

Exercise 11 — Hosted Service

Register:

```
C#
OutboxBackgroundService
```

using:

```
C#
AddHostedService<OutboxBackgroundService>();
```

verify it starts with the host and receives shutdown cancellation.

Exercise 12 — Graceful Shutdown

Inject:

into a demonstration hosted service and register callbacks for:

EXAMPLE / FLOW

```
ApplicationStarted

ApplicationStopping

ApplicationStopped
```

Observe their order when stopping the application normally.

PART 4 — EXACTLY 25 INTERVIEW QUESTIONS

1. What is ASP.NET Core?

ASP.NET Core is Microsoft's cross-platform web framework built on .NET for creating HTTP-based applications such as Web APIs, MVC applications, Minimal APIs, SignalR services, and other web workloads.

Its hosting infrastructure includes:

EXAMPLE / FLOW

```
HTTP server

Dependency injection

Configuration

Logging

Middleware

Routing

Application lifetime
```

Current templates use `WebApplicationBuilder` and `WebApplication` as the recommended hosting model.

2. What is Program.cs in an ASP.NET Core application?

`Program.cs` is normally the application's entry point and composition root.

It typically performs two major tasks:

EXAMPLE / FLOW

```
Before Build
    →
configure host/services

After Build
    →
configure HTTP pipeline/endpoints
```

Modern C# templates use top-level statements, so an explicit `Main` method usually isn't visible.

3. What does WebApplication.CreateBuilder(args) do?

It creates a preconfigured `WebApplicationBuilder`.

The builder exposes facilities for:

EXAMPLE / FLOW

```
Configuration

Environment

Dependency injection

Logging

Host configuration

Web server configuration
```

and applies framework defaults used by modern ASP.NET Core applications.

4. What is WebApplicationBuilder?

`WebApplicationBuilder` is the startup configuration object used before the application is built.

Important members include:

C#

```
builder.Services

builder.Configuration

builder.Environment

builder.Logging

builder.Host

builder.WebHost
```

It is where the application describes the services and hosting configuration required at runtime.

5. What does builder.Build() do?

`builder.Build()` constructs the configured `WebApplication` and its hosting/service infrastructure.

After `Build`, application startup normally moves from:

EXAMPLE / FLOW

```
service registration
```

to:

EXAMPLE / FLOW

```
middleware and endpoint configuration
```

The built application is then started using `Run` or `RunAsync`.

6. What does app.Run() do?

`app.Run()` starts the application host and waits until shutdown.

During startup, hosted services are started, the HTTP pipeline is finalized, and the configured web server—normally Kestrel—begins listening for incoming requests.

7. What is the Generic Host?

The Generic Host provides common application infrastructure for:

EXAMPLE / FLOW

```
Dependency injection

Configuration

Logging

Hosted services

Startup

Graceful shutdown
```

ASP.NET Core's modern web hosting model builds upon the same hosting concepts while exposing the convenient `WebApplicationBuilder` / `WebApplication` API.

8. What is the difference between WebApplication and the older WebHost?

`WebApplication` represents the modern minimal-hosting model used by current ASP.NET Core templates.

The older WebHost/ `Startup` model remains primarily for compatibility with older applications.

For new applications, `WebApplicationBuilder` and `WebApplication` are the normal starting point.

9. What is Kestrel?

Kestrel is ASP.NET Core's cross-platform HTTP web server.

It:

EXAMPLE / FLOW

```
opens network sockets

listens on configured endpoints

accepts HTTP connections

passes requests into ASP.NET Core

sends responses
```

It is enabled by default in current ASP.NET Core project templates.

10. What is the difference between AddControllers and MapControllers?

`AddControllers()` :

EXAMPLE / FLOW

```
registers controller-related services with DI
```

and belongs in the builder/service-registration phase.

`MapControllers()` :

EXAMPLE / FLOW

```
maps controller endpoints into the application
```

and belongs after the application is built.

Conceptually:

EXAMPLE / FLOW

```
AddControllers
    →
make controller infrastructure available

MapControllers
    →
expose controller routes
```

11. What is IConfiguration?

`IConfiguration` is ASP.NET Core/.NET's unified key-value configuration abstraction.

It can combine data from multiple providers such as:

EXAMPLE / FLOW

```
JSON

Environment variables

User Secrets

Command-line arguments

External providers
```

The application reads the resulting merged configuration rather than depending directly on one file.

12. What is the default configuration precedence?

For normal modern ASP.NET Core app configuration, higher-priority values generally follow:

EXAMPLE / FLOW

```
Command Line
    ↓
Environment Variables
    ↓
User Secrets in Development
    ↓
appsettings.{Environment}.json
    ↓
appsettings.json
```

If several sources contain the same key, the higher-priority value wins.

13. What is appsettings.json?

`appsettings.json` is a conventional JSON configuration source used by ASP.NET Core applications.

It may contain non-secret application settings such as:

EXAMPLE / FLOW

```
Logging configuration

Integration URLs

Timeouts

Feature settings
```

```
Connection-string placeholders
```

Environment-specific files can override its values.

14. What are appsettings.Development.json and appsettings.Production.json?

They are environment-specific configuration files.

For example:

EXAMPLE / FLOW

```
Development environment
    →
appsettings.json
+
appsettings.Development.json
```

Matching values in the environment-specific file override the base `appsettings.json` values.

15. What are ASP.NET Core runtime environments?

Runtime environments allow startup/configuration behavior to vary according to where the application is running.

Conventional names are:

EXAMPLE / FLOW

```
Development

Staging

Production
```

Custom values are possible.

If no environment is configured, ASP.NET Core defaults to Production.

16. What is launchSettings.json?

`launchSettings.json` contains local development launch profiles.

It can define:

EXAMPLE / FLOW

```
Local URLs

Environment variables

Browser launch behavior

Development profiles
```

It is local tooling configuration and is not published as production application configuration.

17. What is the Options pattern?

The Options pattern binds related configuration into strongly typed classes.

Instead of:

```
C#
configuration[
    "Receipt:TimeoutSeconds"]
```

consumers can depend on:

```
C#
IOptions<
    ReceiptOptions>
```

This improves encapsulation, type safety, and separation of concerns.

18. What does ValidateOnStart do?

`ValidateOnStart()` causes configured Options validation to execute during application startup.

For mandatory configuration, this allows the application to fail immediately rather than discovering the invalid value on the first request.

19. Where should application secrets be stored?

Production secrets should not be stored directly in source-controlled `appsettings.json` files.

For development, .NET's Secret Manager can keep local secrets outside the project.

For production, use secure external secret-management/configuration mechanisms appropriate to the deployment environment.

20. What is ILogger<T>?

`ILogger<T>` is the standard .NET logging abstraction using `T` as a logging category.

Example:

```
C#
_logger.LogInformation(
    "Gift {GiftId} completed",
    giftId);
```

ASP.NET Core provides logging configuration, filtering, and multiple built-in providers.

21. What is structured logging?

Structured logging stores named values as properties rather than formatting all data into an opaque string.

Prefer:

```
C#
_logger.LogInformation(
    "Gift {GiftId} completed for Tenant {TenantId}",
    giftId,
    tenantId);
```

This allows a logging platform to query:

```
EXAMPLE / FLOW
GiftId
TenantId
```

as structured fields.

22. What is IHostApplicationLifetime?

`IHostApplicationLifetime` allows application code to observe application lifecycle events.

It exposes:

EXAMPLE / FLOW

```
ApplicationStarted

ApplicationStopping

ApplicationStopped
```

and:

C#

```
StopApplication()
```

for requesting graceful shutdown.

23. What is IHostedService?

`IHostedService` represents a component whose lifetime is managed by the host.

The host invokes its startup and shutdown methods as part of application lifecycle management.

ASP.NET Core commonly uses hosted services for:

EXAMPLE / FLOW

```
Background workers

Queue consumers

Outbox processing

Scheduled processing
```

`BackgroundService` is a convenient base class for long-running hosted-service implementations.

24. Why should Program.cs not contain business logic?

`Program.cs` is the composition/startup boundary.

Its responsibility is to configure:

EXAMPLE / FLOW

```
Services

Infrastructure

Configuration

Logging

Middleware

Endpoints
```

Business logic belongs in:

EXAMPLE / FLOW

```
Domain

Application services

Use-case handlers
```

Putting business operations in startup couples application behavior to hosting and makes testing/lifecycle management harder.

25. How would startup work in the Fundraising Platform?

A practical startup flow is:

EXAMPLE / FLOW

```
Process starts
    ↓
WebApplication.CreateBuilder
    ↓
Load:
    appsettings
    environment settings
    secrets
    environment variables
    command line
    ↓
Determine environment
    ↓
Register Controllers
    ↓
Register Application services
    ↓
Register DbContext
    ↓
Register repositories
    ↓
Register integrations
    ↓
Bind + validate Options
    ↓
Register Outbox background worker
    ↓
builder.Build()
    ↓
Configure HTTP middleware
    ↓
Map controllers
    ↓
app.Run()
    ↓
Hosted services start
    ↓
Kestrel listens
    ↓
Fundraising API accepts requests
```

A clean `Program.cs` might therefore communicate:

EXAMPLE / FLOW

```
Application capabilities

Infrastructure dependencies
```

```
Security setup

HTTP pipeline

Endpoint exposure
```

without containing Gift business rules itself.

QUICK REVISION

Concept	Meaning
ASP.NET Core	.NET web framework
`Program.cs`	Entry point/composition root
Top-level statements	Concise compiler-generated `Main`
`WebApplicationBuilder`	Pre-build configuration object
`builder.Services`	DI registrations
`builder.Configuration`	Configuration system
`builder.Environment`	Runtime environment
`builder.Logging`	Logging configuration
`builder.Build()`	Builds `WebApplication`
`WebApplication`	Built web application
`app.Run()`	Starts host and waits for shutdown
Generic Host	DI/config/logging/lifetime infrastructure
Kestrel	Default ASP.NET Core web server
`appsettings.json`	Base JSON configuration
Environment JSON	Environment-specific overrides
Environment variables	Deployment configuration
Command-line config	High-priority configuration
User Secrets	Development secret storage
Options pattern	Strongly typed configuration
`ValidateOnStart`	Fail startup on invalid options
`ILogger<T>`	Typed logging abstraction
Environment	Development/Staging/Production/etc.
`IHostedService`	Host-managed background component
`IHostApplicationLifetime`	Startup/shutdown notifications
Graceful shutdown	Controlled application termination

FINAL END-TO-END CONCEPT FLOW

EXAMPLE / FLOW

```
OPERATING SYSTEM / CONTAINER
          ↓
Starts .NET process
          ↓
Program.cs
          ↓
WebApplication.CreateBuilder(args)
          ↓
┌──────────────────────────────┐
│ HOSTING INFRASTRUCTURE       │
│                              │
│ IConfiguration               │
│ IServiceCollection           │
│ Logging                      │
│ IWebHostEnvironment          │
│ Kestrel configuration        │
│ Hosted services              │
└──────────────────────────────┘
          ↓
Register Fundraising services
          ↓
Register DbContext
          ↓
Register repositories
          ↓
Register integrations
          ↓
Bind / validate configuration
          ↓
builder.Build()
          ↓
WebApplication
          ↓
Configure HTTP pipeline
          ↓
Map Fundraising endpoints
          ↓
app.Run()
          ↓
Hosted services StartAsync
          ↓
Kestrel starts listening
          ↓
ApplicationStarted
          ↓
──────────────────────────────
        RUNNING API
──────────────────────────────
          ↓
HTTP requests arrive
          ↓
ASP.NET Core pipeline
          ↓
Endpoint / Controller
          ↓
Application Layer
          ↓
Domain
          ↓
Infrastructure
```

```
          ↓
Response
```

Shutdown:

EXAMPLE / FLOW

```
SIGTERM / Ctrl+C / Deployment
          ↓
ApplicationStopping
          ↓
Stop accepting new work
          ↓
Finish/cancel managed work
          ↓
Hosted services StopAsync
          ↓
Dispose DI-owned resources
          ↓
ApplicationStopped
          ↓
Process exits
```

The central lesson is:

ASP.NET Core startup is the process of constructing and configuring the application's host. `WebApplicationBuilder` gathers configuration, services, environment, logging, and server settings; `Build()` creates the application and dependency graph; the built `WebApplication` defines the HTTP pipeline and endpoints; and `Run()` starts hosted services and Kestrel and keeps the host alive until graceful shutdown. `Program.cs` should therefore describe how the system is assembled—not contain the business behavior the system performs.

Next roadmap topic: Topic 38 — ASP.NET Core Request Pipeline, Middleware, Routing and Error Handling.